Lab 13A

Role-Based Access Control with API Scopes

WSO2 API Manager 4.7.0 · Practical exercise

Before You Begin

URLs in this guide use placeholders. Replace them with the values of the API Manager you are using:

- **`<APIM_HOST>`** – host name or IP address of the API Manager server (localhost when API Manager runs on your own laptop).
- **`<HTTPS_PORT>`** – API Manager HTTPS port: 9443 plus any port offset (Publisher, Developer Portal, Admin Portal, Management Console, token endpoint).
- **`<GW_HTTPS_PORT>` / `<GW_HTTP_PORT>`** – API Gateway ports: 8243 (HTTPS) and 8280 (HTTP) plus any port offset.
- **`<APIM_HOME>`** – the folder where API Manager is extracted.

The servers use a self-signed certificate by default, so the browser shows a security warning the first time: choose Advanced and continue.

Training Objective

Learn how to allow or block individual API resources by user role. You create roles and users, add scopes to an API, bind each scope to roles, and then check with tokens for different users that the gateway only lets each user call what their role allows.

Business Scenario

PizzaShack opens an internal API for its branch staff. Every staff member can see the menu, cashiers and managers can place and view orders, but only managers can cancel an order. Access must follow the role of the logged-in user, not the application they use.

High-Level Steps

- Create the roles and users
- Create the PizzaShackStaff API
- Add scopes and bind them to roles and resources
- Create an application and generate keys
- Get a token for each user and call the API
- Change a role and see the effect (optional)

How It Works

A scope is a named permission on an API resource. Each scope is bound to one or more roles. When an application asks for a token on behalf of a user, the token only receives the scopes whose roles that user has. At runtime the gateway compares the scope of the resource with the scopes in the token and rejects the call with HTTP 403 if the scope is missing.

CC BY 4.0

Resource	Scope	Roles allowed
`GET /menu`	none	Any user with a valid token
`POST /order`	`order_create`	pizza_staff, pizza_manager
`GET /order/{orderId}`	`order_create`	pizza_staff, pizza_manager
`DELETE /order/{orderId}`	`order_delete`	pizza_manager

Part 1 – Create the Roles and Users

1. Open the Management Console and log in as admin:

```
https://<APIM_HOST>:<HTTPS_PORT>/carbon
```

Example:

```
https://localhost:9443/carbon
```

2. Go to **Main > Users and Roles > Add > Add New Role**, keep the domain **PRIMARY**, enter `pizza_staff` as the role name and click **Finish**. No permissions are needed: these roles are only used for API scopes, not for portal access.
3. Repeat step 2 to create the role `pizza_manager`.
4. Go to **Add > Add New User** and create the users below. Click **Next**, select the role and click **Finish**.

User name	Password	Role
`cashier1`	`password`	`pizza_staff`
`manager1`	`password`	`pizza_manager`
`guest1`	`password`	none (do not select a role)

Part 2 – Create the API

5. Log in to the Publisher as admin:

```
https://<APIM_HOST>:<HTTPS_PORT>/publisher
```

Example:

```
https://localhost:9443/publisher
```

6. Click **Create API**, and under **REST API** select **Start From Scratch**. Enter the values below and click **Create**.

Field	Value
Name	`PizzaShackStaff`
Context	`/pizzashack-staff`
Version	`1.0.0`
Endpoint	The URL in the box below (the PizzaShack sample back end that ships with API Manager)

CC BY 4.0

Endpoint

```
https://localhost:<HTTPS_PORT>/am/sample/pizzashack/v1/api/
```

Example – Endpoint:

```
https://localhost:9443/am/sample/pizzashack/v1/api/
```

Keep `localhost` in the endpoint: the gateway calls the back end on the same server. On a shared server, add your initials to the name and context (for example `PizzaShackStaff-JD` and `/pizzashack-staff-jd`).

7. Go to **API Configurations > Resources**. Delete the default `/*` resources, then add the four resources below with **Add New Resource** (select the HTTP verb, enter the URI pattern and click **+**). Click **Save**.

HTTP verb	URI pattern
GET	`/menu`
POST	`/order`
GET	`/order/{orderId}`
DELETE	`/order/{orderId}`

Part 3 – Add Scopes and Bind Them to Roles

8. Go to **API Configurations > Local Scopes** and click **Create Scopes**. Enter the values below. In **Roles**, type each role name and press Enter. Click **Save**.

Name	Display Name	Description	Roles
`order_create`	Create Orders	Place and view orders	`pizza_staff`, `pizza_manager`

9. Click **Create Scopes** again and add the second scope:

Name	Display Name	Description	Roles
`order_delete`	Cancel Orders	Cancel orders	`pizza_manager`

A role must exist before it can be bound to a scope. If you mistype it, the Publisher shows that the role is not valid.

10. Go back to **API Configurations > Resources**. Expand each resource below and, under **Operation Governance**, select the **Operation scope**. Leave `GET /menu` without a scope. Click **Save**.

Resource	Operation scope
`POST /order`	`order_create`
`GET /order/{orderId}`	`order_create`
`DELETE /order/{orderId}`	`order_delete`

CC BY 4.0

11. Go to **Deploy > Deployments** and click **Deploy** to deploy a revision to the default gateway.

12. Go to **Lifecycle** and click **Publish**.

Part 4 – Create an Application and Generate Keys

13. Log in to the Developer Portal as admin:

```
https://<APIM_HOST>:<HTTPS_PORT>/devportal
```

Example:

```
https://localhost:9443/devportal
```

14. Go to **Applications > Add New Application**, enter `PizzaStaffApp` as the name and click **Save**.

15. Open **Subscriptions** in the application, click **Subscribe APIs** and subscribe to **PizzaShackStaff** with the **Unlimited** business plan.

16. Open **Production Keys > OAuth2 Tokens**. Under **Grant Types**, select **Password** and **Client Credentials**, and click **Generate Keys**. Copy the **Consumer Key** and **Consumer Secret**.

    The Password grant lets the application get a token on behalf of a user, so the token carries that user's roles. In real applications the user usually logs in through a browser (Authorization Code grant); the scope check works the same way.

Part 5 – Get a Token for Each User

17. Request a token for `cashier1` and ask for both scopes. Replace the consumer key and secret with your values:

macOS / Linux

```
curl -k -u "<CONSUMER_KEY>:<CONSUMER_SECRET>" \
  -d "grant_type=password" -d "username=cashier1" -d "password=password" \
  -d "scope=order_create order_delete" \
  "https://<APIM_HOST>:<HTTPS_PORT>/oauth2/token"
```

Example – macOS / Linux:

```
curl -k -u "Xb3fK9qLm2:p7TzR4wYc1" \
  -d "grant_type=password" -d "username=cashier1" -d "password=password" \
  -d "scope=order_create order_delete" \
  "https://localhost:9443/oauth2/token"
```

Windows (Command Prompt)

```
curl -k -u "<CONSUMER_KEY>:<CONSUMER_SECRET>" ^
  -d "grant_type=password" -d "username=cashier1" -d "password=password" ^
  -d "scope=order_create order_delete" ^
  "https://<APIM_HOST>:<HTTPS_PORT>/oauth2/token"
```

Example – Windows (Command Prompt):

```
curl -k -u "Xb3fK9qLm2:p7TzR4wYc1" ^
  -d "grant_type=password" -d "username=cashier1" -d "password=password" ^
  -d "scope=order_create order_delete" ^
  "https://localhost:9443/oauth2/token"
```

CC BY 4.0

Look at the scope field in the response. cashier1 asked for both scopes but only got the one its role allows:

```
> {"access_token":"eyJ4NXQiOi...","scope":"order_create",
>  "token_type":"Bearer","expires_in":3600}
```

18. Repeat step 17 for `manager1` and `guest1` (change only the user name). Save the three access tokens.

User	Role	scope in the response
`cashier1`	`pizza_staff`	`order_create`
`manager1`	`pizza_manager`	`order_create order_delete`
`guest1`	none	`default`

`default` means the token has none of the requested scopes. The token is still valid, so resources without a scope can be called.

Part 6 – Call the API with Each Token

19. Call `GET /menu` with each of the three tokens. This resource has no scope, so all three get **HTTP 200** and the menu.

macOS / Linux

```
curl -k -H "Authorization: Bearer <ACCESS_TOKEN>" \
  "https://<APIM_HOST>:<GW_HTTPS_PORT>/pizzashack-staff/1.0.0/menu"
```

Example – macOS / Linux:

```
curl -k -H "Authorization: Bearer eyJ4NXQiOi.eyJzdWIi.ZXhhbXBsZQ" \
  "https://localhost:8243/pizzashack-staff/1.0.0/menu"
```

Windows (Command Prompt)

```
curl -k -H "Authorization: Bearer <ACCESS_TOKEN>" ^
  "https://<APIM_HOST>:<GW_HTTPS_PORT>/pizzashack-staff/1.0.0/menu"
```

Example – Windows (Command Prompt):

```
curl -k -H "Authorization: Bearer eyJ4NXQiOi.eyJzdWIi.ZXhhbXBsZQ" ^
  "https://localhost:8243/pizzashack-staff/1.0.0/menu"
```

20. Save the order below as a file named `order.json` in the folder where you run curl:

```
{
  "customerName": "Amina",
  "pizzaType": "Chicken Parmesan",
  "quantity": 2,
  "address": "Nairobi",
  "creditCardNumber": "4111111111111111",
  "delivered": false
}
```

`4111111111111111` is a standard test card number, not a real card.

CC BY 4.0

21. Place an order with each token:

macOS / Linux

```
curl -k -X POST -H "Authorization: Bearer <ACCESS_TOKEN>" \
  -H "Content-Type: application/json" -d @order.json \
  "https://<APIM_HOST>:<GW_HTTPS_PORT>/pizzashack-staff/1.0.0/order"
```

Example – macOS / Linux:

```
curl -k -X POST -H "Authorization: Bearer eyJ4NXQiOi.eyJzdWIi.ZXhhbXBsZQ" \
  -H "Content-Type: application/json" -d @order.json \
  "https://localhost:8243/pizzashack-staff/1.0.0/order"
```

Windows (Command Prompt)

```
curl -k -X POST -H "Authorization: Bearer <ACCESS_TOKEN>" ^
  -H "Content-Type: application/json" -d @order.json ^
  "https://<APIM_HOST>:<GW_HTTPS_PORT>/pizzashack-staff/1.0.0/order"
```

Example – Windows (Command Prompt):

```
curl -k -X POST -H "Authorization: Bearer eyJ4NXQiOi.eyJzdWIi.ZXhhbXBsZQ" ^
  -H "Content-Type: application/json" -d @order.json ^
  "https://localhost:8243/pizzashack-staff/1.0.0/order"
```

cashier1 and manager1 get **HTTP 201** with the new order. Copy the `orderId` from the response. guest1 is rejected:

```
> {"code":"900910","message":"The access token does not allow you to access
>  the requested resource","description":"User is NOT authorized to access the
>  Resource: /order. Scope validation failed."}
```

22. Cancel the order with the `cashier1` token, then with the `manager1` token. Replace `<ORDER_ID>` with the orderId you copied:

macOS / Linux

```
curl -k -X DELETE -H "Authorization: Bearer <ACCESS_TOKEN>" \
  "https://<APIM_HOST>:<GW_HTTPS_PORT>/pizzashack-staff/1.0.0/order/<ORDER_ID>"
```

Example – macOS / Linux:

```
curl -k -X DELETE -H "Authorization: Bearer eyJ4NXQiOi.eyJzdWIi.ZXhhbXBsZQ" \
  "https://localhost:8243/pizzashack-staff/1.0.0/order/9f6f8aa4-ce7c-41d6-b263-852660f8bf81"
```

Windows (Command Prompt)

```
curl -k -X DELETE -H "Authorization: Bearer <ACCESS_TOKEN>" ^
  "https://<APIM_HOST>:<GW_HTTPS_PORT>/pizzashack-staff/1.0.0/order/<ORDER_ID>"
```

Example – Windows (Command Prompt):

```
curl -k -X DELETE -H "Authorization: Bearer eyJ4NXQiOi.eyJzdWIi.ZXhhbXBsZQ" ^
  "https://localhost:8243/pizzashack-staff/1.0.0/order/9f6f8aa4-ce7c-41d6-b263-852660f8bf81"
```

CC BY 4.0

cashier1 gets **HTTP 403** (code `900910`, Scope validation failed) and manager1 gets **HTTP 200**. The results should match this table:

Call	cashier1	manager1	guest1
`GET /menu`	200	200	200
`POST /order`	201	201	403 (900910)
`DELETE /order/{orderId}`	403 (900910)	200	403 (900910)

Part 7 – Change a Role and See the Effect (Optional)

23. In the Management Console, go to **Main > Users and Roles > List > Users**. Next to `cashier1`, click **Assign Roles**, select `pizza_manager` and click **Update**.

24. Place a new order (step 21) and try to cancel it with the old cashier1 token (step 22). It is still rejected with **HTTP 403**: the scopes of a token are fixed when the token is issued.

25. Get a new token for cashier1 (step 17). The scope is now `order_create order_delete`, and cancelling the order with the new token returns **HTTP 200**.

Access is given to roles, not to users. Changing a user's roles takes effect from the next token the user gets.

Expected Outcome

The PizzaShackStaff API has two scopes bound to roles. Every user can read the menu, only pizza_staff and pizza_manager can place and view orders, and only pizza_manager can cancel them. Tokens only receive the scopes allowed by the user's roles, and the gateway rejects calls without the right scope with HTTP 403 (code 900910).

CC BY 4.0